National Technical University of Athens

Neurofuzzy Control and Applications

Model Predictive Control and Approximate Dynamic Programming for a Planar Car

Semester Project

Petros Georgoulas

March 2023

---

1 Dynamics of a simple system and solution using MPC

Consider a system with the following dynamics, $\boldsymbol{x}_{k+1} = f(\boldsymbol{x}_k, \boldsymbol{u_k})$:

$$\boldsymbol{x}_{k+1} = \begin{bmatrix} x_{k+1} \\ y_{k+1} \\ \theta_{k+1} \end{bmatrix} = \begin{bmatrix} x_k + \Delta t \cdot a\cos(\theta_k) \cdot u_k^1 \\ y_k + \Delta t \cdot a\sin(\theta_k) \cdot u_k^1 \\ \theta_k + \Delta t \cdot b \cdot u_k^2 \end{bmatrix} \tag{1.1}$$

Our objective is to drive the vehicle governed by these equations to its destination with a specified orientation, a state denoted hereafter by $\boldsymbol{x}_{ref}$, while minimizing the total cost. To achieve this, we use the dynamic programming equations:

$$\begin{aligned} J_N(\boldsymbol{x}_N) &= g_N(\boldsymbol{x}_N) \\ J_k(\boldsymbol{x}_k) &= \min_{\boldsymbol{u}_k} \left\{ g_k(\boldsymbol{x}_k, \boldsymbol{u}_k) + J_{k+1}(f_k(\boldsymbol{x}_k, \boldsymbol{u}_k)) \right\} \end{aligned} \tag{1.2}$$

For the terminal cost $J_N(\boldsymbol{x}_N)$, we assume that

$$J_N(\boldsymbol{x}_N) = g_N(\boldsymbol{x}_N) = (\boldsymbol{x}_N - \boldsymbol{x}_{ref})^T W_s (\boldsymbol{x}_N - \boldsymbol{x}_{ref}) \tag{1.3}$$

where W_s is a diagonal matrix that assigns a different weight to each variable. We also assume that

$$g_k(\boldsymbol{x}_k, \boldsymbol{u}_k) = g(\boldsymbol{x}_k, \boldsymbol{u}_k) = (\boldsymbol{x}_k - \boldsymbol{x}_{ref})^T W_s (\boldsymbol{x}_k - \boldsymbol{x}_{ref}) + \boldsymbol{u}_k^T W_c \boldsymbol{u}_k \tag{1.4}$$

To solve the dynamic programming equation, we must discretize the state space (x_k, y_k, θ_k), as shown in the following figure. Each point corresponds to a different state (x_k, y_k, θ_k).

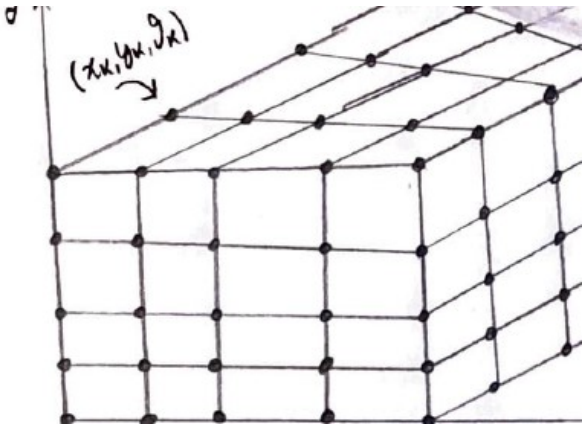


Figure 1: Discretization of the state space

To solve this problem, we use **multistep lookahead and rolling horizon**. Specifically, we set

$$\tilde{J}_{k+l}(\boldsymbol{x}_{k+l}) = g_N(\boldsymbol{x}_{k+l}) \tag{1.5}$$

where g_N is defined in equation (1.3). The idea is to treat this as a standard dynamic programming problem, but to consider only l steps at a time. Specifically, for each initial state $\boldsymbol{x}_k$, we follow the procedure below.

1. Compute $J_{k+l}(\boldsymbol{x}_{k+l}), \forall \boldsymbol{x}_{k+l}$, using equation (1.3).

2. $\forall \boldsymbol{x}_{k+l-1}$, solve
$$J_{k+l-1}(\boldsymbol{x}_{k+l-1}) = \min_{\boldsymbol{u}_{k+l-1}} \Big\{ g_{k+l-1}(\boldsymbol{x}_{k+l-1}) + J_{k+l}(\boldsymbol{x}_{k+l}) \Big\}$$
For the resulting control $\boldsymbol{u}_{k+l-1}$, the state $\boldsymbol{x}_{k+l}$ will generally not coincide with a grid point in Figure 1. We therefore need to interpolate the function $J_{k+l}(\boldsymbol{x}_{k+l})$.

3. Continue this procedure, computing J at every point, until $l-1$ steps have been completed. Once $J_{k+1}(\boldsymbol{x}_{k+1})$ is known $\forall \boldsymbol{x}_{k+1}$, compute the vector $\boldsymbol{u}_k$ that minimizes the right-hand side of equation (1.2), and hence the next state $\boldsymbol{x}_{k+1} = f_k(\boldsymbol{x}_k, \boldsymbol{u}_k)$.

4. Repeat this procedure for as many steps as necessary, until the difference $\|\boldsymbol{x}_k - \boldsymbol{x}_{ref}\|_W$ falls below a specified threshold.

Why recompute $\mathbf{J_{k+1}}, \mathbf{J_{k+2}}, \ldots$ at every step? Since the problem is deterministic, it could theoretically be solved using exact dynamic programming. That is, we could compute all the functions $J_k, J_{k+1}, \ldots$ (possibly with a discount factor), and then apply the controls $u_k, u_{k+1}, \ldots$. Furthermore, since the destination is fixed, the problem is static, so the functions J are independent of time k. Nevertheless, the formulation adopted here also makes it straightforward to solve the problem when the destination moves as a function of time k, without modifying our algorithms.

1.1 Simulation

The following examples use different initial states $\boldsymbol{x}_0$, desired terminal states $\boldsymbol{x}_{ref}$, and lookahead values l.
In every case, we set $\Delta t = 0.4$ and $a = b = 0.5$ in equation (1.1), which governs the system dynamics. In equation (1.4), we also set

$$W_s = \begin{bmatrix} 5 & 0 & 0 \\ 0 & 5 & 0 \\ 0 & 0 & 0.2 \end{bmatrix} \qquad W_c = \begin{bmatrix} 1 & 0 \\ 0 & 0.2 \end{bmatrix}$$

The matrix W_s has this form because we place greater importance on reaching the desired terminal position (x_{ref}, y_{ref}) than on reaching it with the correct orientation θ_{ref}. We also assume that

$$0 \leq x \leq 6 \quad 0 \leq y \leq 6 \quad 0 \leq \theta \leq \pi$$

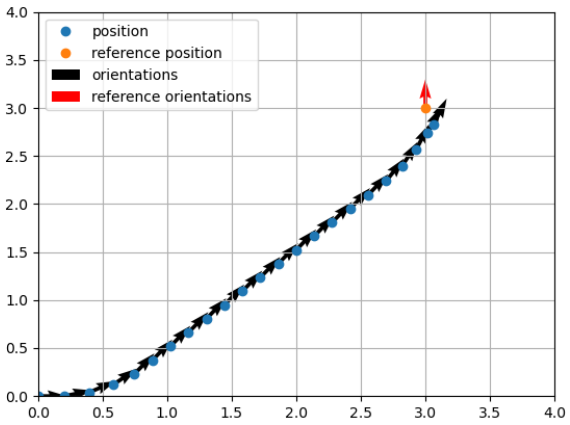
$$-1 \leq u_1 \leq 1 \quad -1 \leq u_2 \leq 1$$

With larger admissible ranges, a much finer discretization was required to obtain satisfactory results, making the computational complexity extremely high. The variables x, y are uniformly discretized into 30 values each, whereas θ, u_1, u_2 are discretized into 5 values each. Finally, to determine when to stop iterating, we use the l^∞ norm, and stop when

$$\|\boldsymbol{x}_{k+1} - \boldsymbol{x}_k\|_\infty \leq \text{threshold}$$

Simulation results for $l = 2$

The following results use two-step lookahead with $\boldsymbol{x}_0 = (0, 0, 0)$ and $\boldsymbol{x}_{ref} = (3, 3, \pi/2)$.



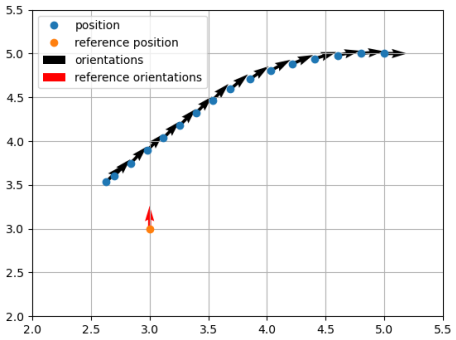
The figure shows that the vehicle does reach a point very close to the desired terminal position, with a relatively good orientation. The difficulty encountered is that, with the chosen function $g_k(\boldsymbol{x}_k, \boldsymbol{u}_k)$, when the vehicle is far from the target position, minimizing the error $||\boldsymbol{x}_k - \boldsymbol{x}_{ref}||_{W_s}$ carries more weight, while consuming a large amount of energy $||\boldsymbol{u}_k||_{W_c}$ carries less. Near the desired terminal state $\boldsymbol{x}_{ref}$, avoiding high energy consumption becomes more important, since $||\boldsymbol{x}_k - \boldsymbol{x}_{ref}||_{W_s}$ is small. More specifically, we have

$$g_k(\boldsymbol{x}_k, \boldsymbol{u}_k) = ||\boldsymbol{e}||_{W_s} + ||\boldsymbol{u}_k||_{W_c} \quad ||\boldsymbol{e}||_{W_s} = ||\boldsymbol{x}_k - \boldsymbol{x}_{ref}||_{W_s} \tag{1.6}$$

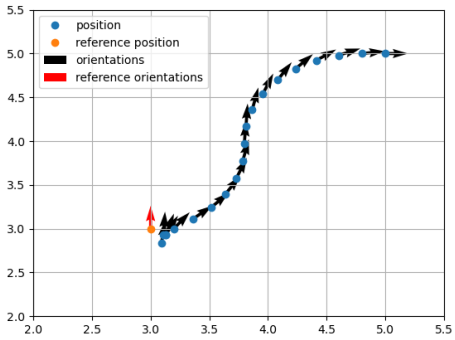
We propose the new cost function

$$g_k(\boldsymbol{x}_k, \boldsymbol{u}_k) = \lambda_1(||\boldsymbol{e}||_{W_s}) \cdot ||\boldsymbol{e}||_{W_s} + \lambda_2(||\boldsymbol{e}||_{W_s}) \cdot ||\boldsymbol{u}_k||_{W_c} \tag{1.7}$$

The functions $\lambda_1(\cdot), \lambda_2(\cdot)$ depend on the norm $||\boldsymbol{e}||_{W_s}$ because, as this norm decreases, we want to place proportionally greater importance on reaching the target, whereas farther away we want to place greater importance on reducing energy consumption. Thus, $\lambda_1(\cdot)$ should be decreasing and $\lambda_2(\cdot)$ increasing. Although there are many possible choices based on this reasoning (indeed, infinitely many), I chose $\lambda_1(x) = 1$ and $\lambda_2(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$. The following results use $\boldsymbol{x}_0 = (5, 5, 0)$ and $\boldsymbol{x}_{ref} = (3, 3, \pi/2)$ for each of equations (1.6) and (1.7).

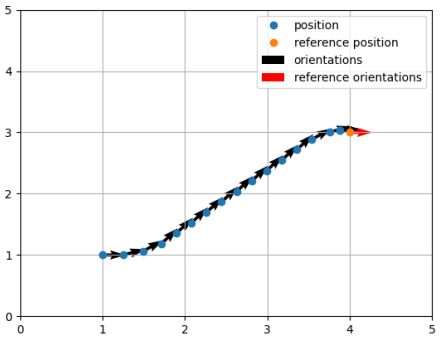


(a) cost function (1.6)

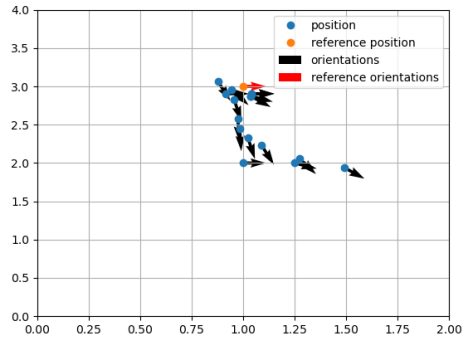


(b) cost function (1.7)

The new cost function indeed produces much better system behavior. Two further examples with different initial and terminal states $\boldsymbol{x}_0, \boldsymbol{x}_{ref}$ are shown below.



(a) $x_0 = (1, 1, 0), x_{ref} = (4, 3, 0)$



(b) $x_0 = (1, 2, 0), x_{ref} = (1, 3, 0)$

Another difficulty with this method is that, when the vehicle approaches its destination with a poor orientation, it sometimes chooses to stop relatively far from the target. This is reasonable under the assumed cost function: stopping relatively far from the target with a good orientation is preferable to consuming considerable fuel to reach the correct position and then correcting the orientation.
This behavior is due to considering only a few steps ahead. For an infinite-horizon problem, the accumulated cost would instead become much larger than the control cost required to reach the correct position, where the cost is zero. To counteract this effect, we follow a procedure similar to the previous one. Specifically, we define an error p associated with the vehicle's **distance** from the terminal position:

$$p = \begin{pmatrix} x_k - x_{ref} & y_k - y_{ref} \end{pmatrix} \cdot \begin{pmatrix} 5 & 0 \\ 0 & 5 \end{pmatrix} \cdot \begin{pmatrix} x_k - x_{ref} \\ y_k - y_{ref} \end{pmatrix}$$

and define the new matrix

$$W_s = \begin{pmatrix} 5 & 0 & 0 \\ 0 & 5 & 0 \\ 0 & 0 & \tanh(p) \end{pmatrix}$$

This matrix ensures that, as we approach the desired terminal state, we place less importance on the desired orientation and more on the position. Another observation is that the farther we are from the target, the less our control $\boldsymbol{u}_k$ affects the final result. Conversely, near the terminal position, our movements must be as refined as possible. This suggests using a short lookahead far from the target, while a substantially longer lookahead may be necessary nearby.

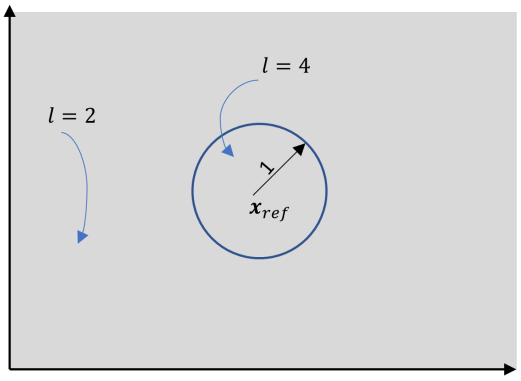


Figure 2: Different lookahead values outside the circle of radius 1 centered at $\boldsymbol{x}_{ref}$

The results are relatively good despite the coarse discretization of the state space (x, y, θ) and the small value of l. However, this method is rather slow, and for larger values of l, which are sometimes necessary, the problem becomes computationally infeasible.

2 Approximation in value-policy space

We next attempt further approximations, since the algorithm used so far solves the problem relatively well but has very high complexity. We therefore consider additional approximations, either of the controls $\boldsymbol{u}$ or of the cost-to-go J_{k+1}.

2.1 Approximating $\tilde{\boldsymbol{\mu}}(\boldsymbol{x})$ with a neural network

So far, we have solved

$$\tilde{J}_k(\boldsymbol{x}_k) = \min_{\boldsymbol{u}_k} \Big\{ g_k(\boldsymbol{x}_k, \boldsymbol{u}_k) + \tilde{J}_{k+1}(f_k(\boldsymbol{x}_k, \boldsymbol{u}_k)) \Big\}$$

for $l-1$ steps at a time, assuming $\tilde{J}_{k+l}(\boldsymbol{x}_{k+l}) = J_N(\boldsymbol{x}_N) = g_N(\boldsymbol{x}_N)$, with the cost given in equation (1.3). Keeping $\boldsymbol{x}_{ref}$ fixed, our objective is to construct a function $\boldsymbol{\mu} : \mathcal{X} \to \mathcal{U}$ that directly provides a control $\boldsymbol{u}_k$ from the current state $\boldsymbol{x}_k$. We hope that this control will resemble the one obtained previously. The function $\boldsymbol{\mu}$ does not depend on time k, since the problem as defined in this subsection is static. To this end, we follow the procedure below.

1. Start from random initial states $\boldsymbol{x}_0$ and use the multistep lookahead algorithm implemented so far to compute the corresponding controls $\boldsymbol{u}_k$. This produces a large collection of samples $(\boldsymbol{x}^s, \boldsymbol{u}^s), s = 1, 2, \ldots, q$, where q is the total number of samples. In every case, the samples $\boldsymbol{u}^s$ are generated as illustrated in Figure 2: two-step lookahead is used outside the circle of radius 1, and four-step lookahead inside it.

2. Approximate the function $\mu(\boldsymbol{x}_k)$ with another function $\tilde{\mu}(\boldsymbol{x}_k, \boldsymbol{r})$, where $\boldsymbol{r}$ is a vector of parameters to be learned. For example, $\tilde{\boldsymbol{\mu}}$ could be a polynomial in $\boldsymbol{x}_k$, or a neural network that takes the state $\boldsymbol{x}$ as input and outputs the control $\boldsymbol{u}$. In either case, we seek the parameter vector $\boldsymbol{r}$ that minimizes the mean squared error between the samples $\boldsymbol{u}^s$ and their predictions $\tilde{\boldsymbol{\mu}}(\boldsymbol{x}^s, \boldsymbol{r})$. Thus, we seek

$$\boldsymbol{r} = \arg\min_{\boldsymbol{r}} \frac{1}{q} \sum_{s=1}^{q} ||\boldsymbol{u}^s - \tilde{\mu}(\boldsymbol{x}^s, \boldsymbol{r})||^2 \tag{2.1}$$

Since the control $\boldsymbol{u}_k$ has two components u_{k1}, u_{k2}, writing

$$\tilde{\boldsymbol{\mu}}(\boldsymbol{x}^s, \boldsymbol{r}) = \begin{pmatrix} \tilde{\mu}_1(\boldsymbol{x}^s, \boldsymbol{r}) \\ \tilde{\mu}_2(\boldsymbol{x}^s, \boldsymbol{r}) \end{pmatrix}$$

allows equation (2.1) to be written as

$$\boldsymbol{r} = \arg\min_{\boldsymbol{r}} \frac{1}{q} \sum_{s=1}^{q} \left[(u_1^s - \tilde{\mu}_1(\boldsymbol{x}^s, \boldsymbol{r}))^2 + (u_2^s - \tilde{\mu}_2(\boldsymbol{x}^s, \boldsymbol{r}))^2 \right] \tag{2.2}$$

We next approximate the vector-valued function $\tilde{\boldsymbol{\mu}}(\boldsymbol{x})$ with a neural network. To train the network, 1584 samples of the form $(\boldsymbol{x}_k^s, \boldsymbol{u}_k^s)$ were generated using the multistep lookahead algorithm, and equation (2.2) was then used. The network has the following structure:

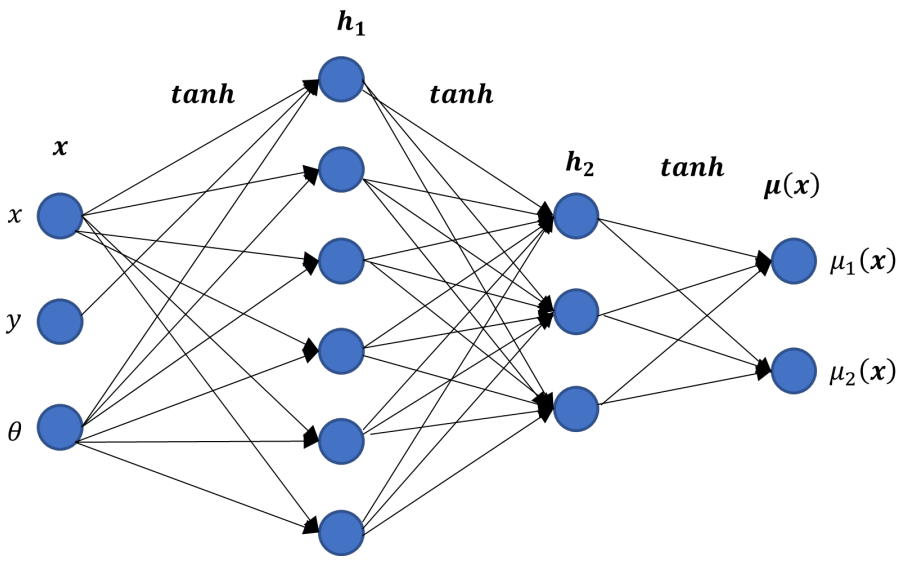


Figure 3: Structure of the neural network used

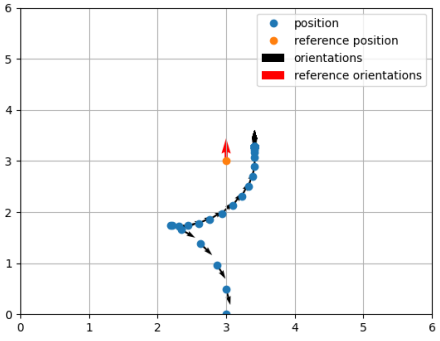
As shown above, the implemented multilayer perceptron has two hidden layers with 6 and 3 neurons, respectively. Thus,

$$\boldsymbol{h}_1 = \tanh(A_1\boldsymbol{x} + \boldsymbol{b}_1)$$

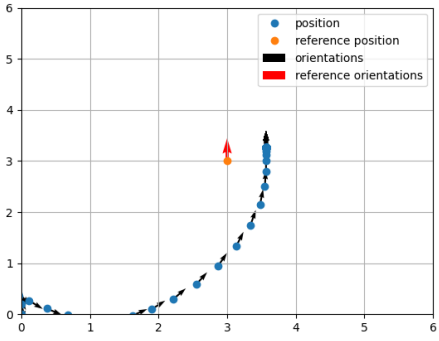
$$\boldsymbol{h}_2 = \tanh(A_2\boldsymbol{h}_1 + \boldsymbol{b}_2)$$

$$\tilde{\boldsymbol{\mu}}(\boldsymbol{x}) = \tanh(A_3\boldsymbol{h}_2 + \boldsymbol{b}_3)$$

The parameters to be learned are the matrices A_1, A_2, A_3 and the biases $\boldsymbol{b}_1, \boldsymbol{b}_2, \boldsymbol{b}_3$. The Adam optimizer was used for 50 epochs over all samples. Assuming that the optimal parameter vector $\boldsymbol{r}_{opt}$ has been found, we have the function $\tilde{\boldsymbol{\mu}}(\boldsymbol{x}, \boldsymbol{r}_{opt})$ available. At each state $\boldsymbol{x}$, it provides a control $\tilde{\boldsymbol{\mu}}$ that we hope will take the vehicle to $\boldsymbol{x}_{ref}$. Some results of this procedure are shown below.

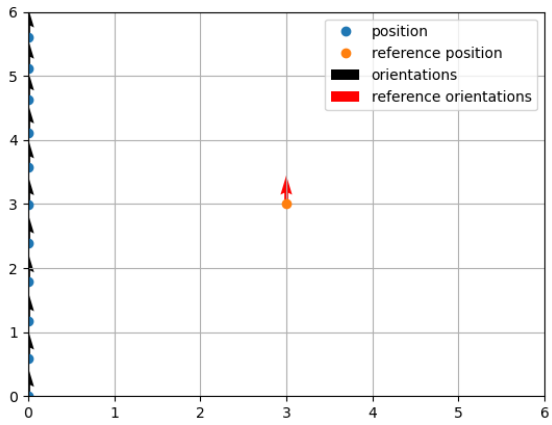


(a) $\boldsymbol{x}_0 = (3, 0, -\pi/2), \boldsymbol{x}_{ref} = (3, 3, \pi/2)$



(b) $\boldsymbol{x}_0 = (0, 0, -\pi/2), \boldsymbol{x}_{ref} = (3, 3, \pi/2)$

Although the algorithm works relatively well for some initial conditions, it performs very poorly for others.



The behavior of the network varied considerably with the activation function. For example, with **ReLU**, the results were very poor regardless of the initial state $\boldsymbol{x}_0$.

2.2 Approximating cost to go with a neural network

We next attempt to approximate the cost-to-go $J_{k+1}(\boldsymbol{x}_{k+1})$ with a neural network. We expect this algorithm to perform substantially better than the previous one, since previously the cost of each state did not affect the decision $\boldsymbol{u}_k$ at all; the decision was based solely on prior experience. Our objective is now to reduce the lookahead to one step. Thus, instead of solving the following problem for each state $\boldsymbol{x}_k$,

$$\tilde{J}_{k+l}(\boldsymbol{x}_{k+l}) = g_N(\boldsymbol{x}_{k+l})$$

$$\tilde{J}_k(\boldsymbol{x}_k) = \min_{\boldsymbol{u}_k}\Big\{g_k(\boldsymbol{x}_k,\boldsymbol{u}_k) + \tilde{J}_{k+1}(\boldsymbol{x}_{k+1})\Big\} \quad k+1, k+2, \ldots, k+l-1$$

we now solve

$$\hat{J}_k(\boldsymbol{x}_k) = \min_{\boldsymbol{u}_k}\Big\{g_k(\boldsymbol{x}_k,\boldsymbol{u}_k) + \hat{J}_{k+1}(\boldsymbol{x}_{k+1})\Big\} \tag{2.3}$$

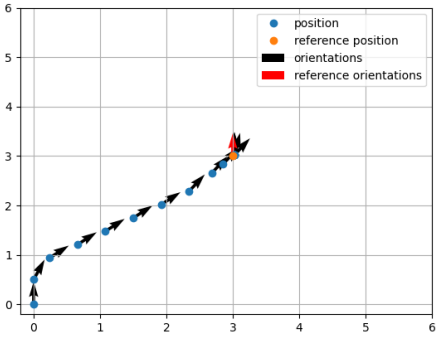
In effect, we attempt *to encode the cost over l steps in a parametric function*, so that at each state $\boldsymbol{x}_k$ we need to perform only one step of dynamic programming (one-step lookahead), rather than 3–4. We hope that $\hat{J}(\boldsymbol{x}_{k+1})$ will closely resemble the function $\tilde{J}(\boldsymbol{x}_{k+1})$ computed previously. Again, $\hat{J}_k = \hat{J}$, since the current formulation of the problem is static.
The reasoning is very similar to that in subsection 2.1, and leads to the following procedure:

1. Generate samples $\boldsymbol{x}^s = (x^s, y^s, \theta^s)$, where $x \in (0,6), y \in (0,6), \theta \in (0,\pi)$, and compute the cost $\beta^s = \tilde{J}(\boldsymbol{x}^s)$ using the multistep lookahead algorithm already implemented. This produces a dataset $(\boldsymbol{x}^s, \beta^s), s = 1, 2, \ldots, q$.

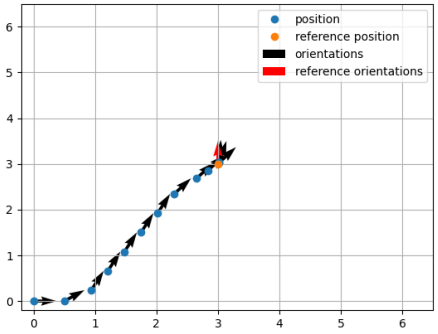
2. Construct a parametric function $\hat{J}(\boldsymbol{x}, \boldsymbol{r})$ that approximates $\tilde{J}(\boldsymbol{x})$ as closely as possible. We therefore seek the parameter vector $\boldsymbol{r}_{opt}$ that minimizes the mean squared error

$$\min_{\boldsymbol{r}} \quad \frac{1}{q}\sum_{s=1}^{q}\Big(\beta^s - \hat{J}(\boldsymbol{x}^s, \boldsymbol{r})\Big)^2$$

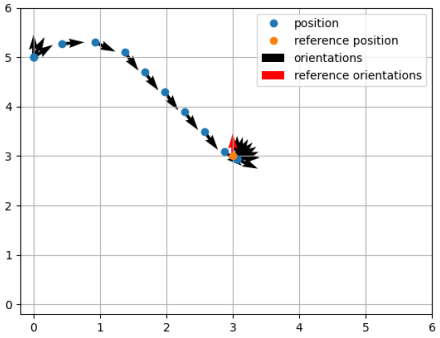
Once $\hat{J}(\boldsymbol{x}, \boldsymbol{r}_{opt})$ has been obtained, we can solve the dynamic programming problem (2.3).
This algorithm produced by far the best results, both in accuracy, as illustrated below, and in execution time. Specifically, it computes an entire trajectory in less than half a second. Four trajectories with different initial states $\boldsymbol{x}_0$ are shown below.



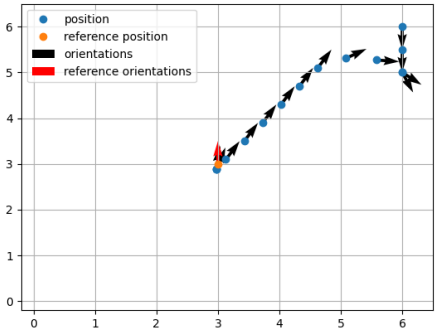
(c) $\boldsymbol{x}_0 = (0, 0, \pi/2), \boldsymbol{x}_{ref} = (3, 3, \pi/2)$



(d) $\boldsymbol{x}_0 = (0, 0, 0), \boldsymbol{x}_{ref} = (3, 3, \pi/2)$



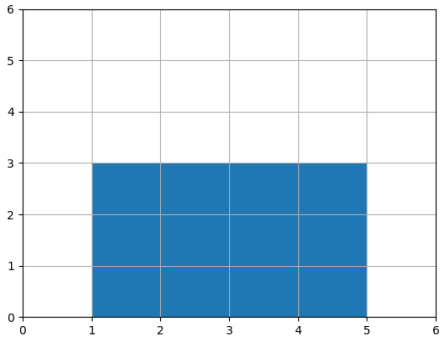
(e) $\boldsymbol{x}_0 = (0, 5, \pi/2), \boldsymbol{x}_{ref} = (3, 3, \pi/2)$



(f) $\boldsymbol{x}_0 = (6, 6, -\pi/2), \boldsymbol{x}_{ref} = (3, 3, \pi/2)$

2.3 Value iteration

In this part, the vehicle must avoid fixed obstacles. Specifically, it cannot move within the region $1 \leq x \leq 5$ and $0 \leq y \leq 3$.



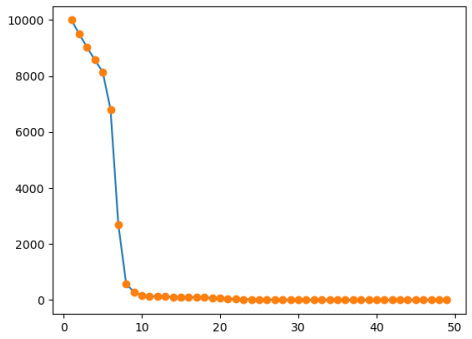
Our objective is now to find the truly optimal control $\mu^*(\boldsymbol{x}) = \begin{pmatrix} u_1 & u_2 \end{pmatrix}^T$ that drives the vehicle to the target. We use value iteration for this purpose. Specifically, we initialize the cost at each state to a random value and update it from J_k to J_{k+1} according to

$$J_{k+1}(\boldsymbol{x}) = \min_{\boldsymbol{u}} \Big\{ g(\boldsymbol{x}, \boldsymbol{u}) + a J_k\big(f(\boldsymbol{x}, \boldsymbol{u})\big) \Big\} \xrightarrow{k \to \infty} J^*(\boldsymbol{x}) \tag{2.4}$$

To avoid the obstacle shown above, we simply assign a very large cost to that region. Although the problem has a finite horizon, since there is a terminal state that will certainly be reached, we introduce a discount factor because the sum of costs at each state may become very large and cause numerical instability. To examine whether the algorithm converges, we consider the maximum difference between two successive iterations:

$$E_k = \max_{\boldsymbol{x} \in \mathcal{X}} \Big\{ J_{k+1}(\boldsymbol{x}) - J_k(\boldsymbol{x}) \Big\} \tag{2.5}$$

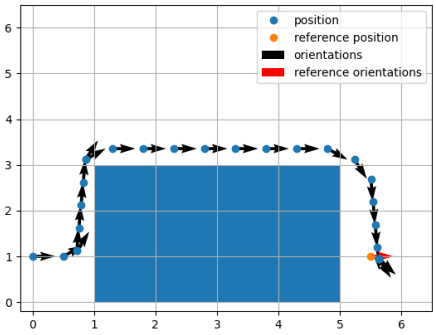
For the following experimental results, we assume $a = 0.95$ and $\boldsymbol{x}_{ref} = (5.5, 1, 0)^T$.



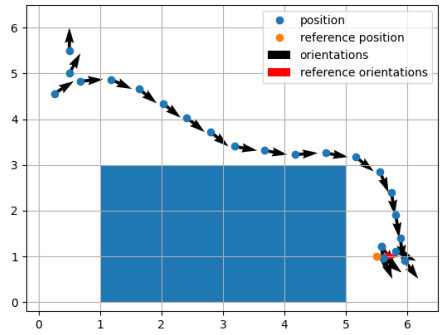
As $E_k \to 0$, we assume that the optimal costs J^* have been found. At each point, it then suffices to solve

$$J^*(\boldsymbol{x}_k) = \min_{u_k} \Big\{ g(\boldsymbol{x}_k, \boldsymbol{u}_k) + a \cdot J^*\big(f(\boldsymbol{x}_k, \boldsymbol{u}_k)\big) \Big\} \tag{2.6}$$

Once J^* has been computed, a relatively expensive operation that must be performed offline, the optimal trajectory can be found very quickly. Some trajectories obtained using this method are shown below.



(g) $\boldsymbol{x}_0 = (0, 1, 0), \boldsymbol{x}_{ref} = (5.5, 1, 0)$



(h) $\boldsymbol{x}_0 = (0.5, 5.5, \pi/2), \boldsymbol{x}_{ref} = (0.5, 5.5, 0)$

The trajectories are now indeed optimal: as shown, the shortest path is followed in each case. This is reasonable, since decisions are now based on J^* rather than an estimate of it.

3 Moving target

3.1 solving the problem using multistep lookahead

We next consider the case in which the target moves with a specified probability depending on the chosen control $\boldsymbol{u}_k$.
The vehicle dynamics are again described by equation (1.1), namely

$$\begin{pmatrix} x_{k+1} \\ y_{k+1} \\ \theta_{k+1} \end{pmatrix} = \begin{pmatrix} x_k \\ y_k \\ \theta_k \end{pmatrix} + \begin{pmatrix} \Delta t \cdot \alpha \cos(\theta_k) \cdot u_k^1 \\ \Delta t \cdot \alpha \sin(\theta_k) \cdot u_k^1 \\ \Delta t \cdot b u_k^2 \end{pmatrix}$$

Suppose that the target point $\boldsymbol{r}$ has dynamics described by the function h_k, so that

$$\boldsymbol{r}_{k+1} = \begin{pmatrix} a_{k+1} \\ b_{k+1} \end{pmatrix} = h_k(\boldsymbol{r}_k, \boldsymbol{u}_k, w_k) \tag{3.1}$$

where w_k is a random variable described below. Whenever the vehicle travels a given distance in a particular direction, the target either remains stationary with probability $1/3$, or moves perpendicular to the direction of travel, in either of the two directions, each with probability $1/3$. When it moves, it travels **the same distance as the vehicle**, that is, $||(x_{k+1}, y_{k+1}) - (x_k, y_k)||_2 = ||\boldsymbol{r}_{k+1} - \boldsymbol{r}_k||_2$.

$$\boldsymbol{y}_k = \begin{pmatrix} \Delta t \cdot \alpha \cos(\theta_k) \cdot u_k^1 \\ \Delta t \cdot \alpha \sin(\theta_k) \cdot u_k^1 \\ \Delta t \cdot b u_k^2 \end{pmatrix} \tag{3.2}$$

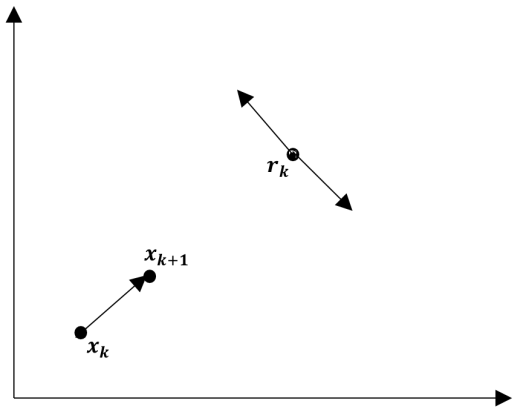


Figure 4

The target dynamics are therefore

$$\boldsymbol{r}_{k+1} = \begin{pmatrix} a_{k+1} \\ b_{k+1} \end{pmatrix} = \begin{pmatrix} a_k \\ b_k \end{pmatrix} + w_k \cdot R\left(\frac{\pi}{2}\right) \begin{pmatrix} \Delta t \cdot a \cos\theta_k \cdot u_k^1 \\ \Delta t \cdot a \sin\theta_k \cdot u_k^1 \end{pmatrix}$$

where $R(\phi)$ is a planar rotation matrix and w_k is uniformly distributed over $\{-1, 0, 1\}$. This includes all cases in which the target remains stationary or moves perpendicular to the direction of travel. To give the state vectors $\boldsymbol{x}_k, \boldsymbol{r}_k$ the same dimension and simplify the calculations, we set

$$\boldsymbol{r}_k = \begin{pmatrix} a_k \\ b_k \\ 0 \end{pmatrix} \qquad R(\phi) = \begin{pmatrix} \cos\phi & -\sin\phi & 0 \\ \sin\phi & \cos\phi & 0 \\ 0 & 0 & 0 \end{pmatrix}$$

Writing the entire increment in equation (1.1) as a vector $\boldsymbol{y}_k$, the equations of the two systems become

$$\begin{aligned} \boldsymbol{x}_{k+1} &= \boldsymbol{x}_k + \boldsymbol{y}_k \\ \boldsymbol{r}_{k+1} &= \boldsymbol{r}_k + w_k \cdot R\left(\frac{\pi}{2}\right) \cdot \boldsymbol{y}_k \end{aligned}$$

The reason for this formulation is that we do not want to include a_k, b_k in the state vector, since the state space would then become too large to solve by discretization. We therefore define the error vector $\boldsymbol{d}_k = \boldsymbol{x}_k - \boldsymbol{r}_k$, giving

$$\begin{aligned} \boldsymbol{d}_{k+1} - \boldsymbol{d}_k &= (\boldsymbol{x}_{k+1} - \boldsymbol{r}_{k+1}) - (\boldsymbol{x}_k - \boldsymbol{r}_k) \\ &= (\boldsymbol{x}_{k+1} - \boldsymbol{x}_k) - (\boldsymbol{r}_{k+1} - \boldsymbol{r}_k) \\ &= \boldsymbol{y}_k - w_k \cdot R\left(\frac{\pi}{2}\right) \boldsymbol{y}_k \Rightarrow \end{aligned}$$

$$\begin{aligned} \boldsymbol{d}_{k+1} &= \boldsymbol{d}_k + \boldsymbol{y}_k - w_k \cdot R\left(\frac{\pi}{2}\right) \boldsymbol{y}_k \\ \boldsymbol{d}_{k+1} &= \boldsymbol{d}_k + \left(I_{3\times 3} - w_k \cdot R\left(\frac{\pi}{2}\right)\right) \cdot \boldsymbol{y}_k \end{aligned} \tag{3.3}$$

Thus, the state vector again contains only three variables: the errors along the x and y axes, and the vehicle angle θ. Substitution gives the following system dynamics:

$$\boldsymbol{d}_{k+1} = \begin{pmatrix} d_{k+1,x} \\ d_{k+1,y} \\ \theta_{k+1} \end{pmatrix} = \begin{pmatrix} d_{k,x} \\ d_{k,y} \\ \theta_k \end{pmatrix} + \left[\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} - w_k \cdot \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix} \right] \cdot \begin{pmatrix} \Delta t \cdot \alpha \cos(\theta_k) \cdot u_k^1 \\ \Delta t \cdot \alpha \sin(\theta_k) \cdot u_k^1 \\ \Delta t \cdot b u_k^2 \end{pmatrix} \tag{3.4}$$

The dynamics are now in the standard form $\boldsymbol{d}_{k+1} = f_k(\boldsymbol{d}_k, \boldsymbol{u}_k, w_k)$. Our objective is to find controls $\boldsymbol{u}_k$ that bring the vehicle to the target. We could therefore propose

$$g_k(\boldsymbol{d}_k) = ||\boldsymbol{d}_k||^2 = d_{k,x}^2 + d_{k,y}^2 \tag{3.5}$$

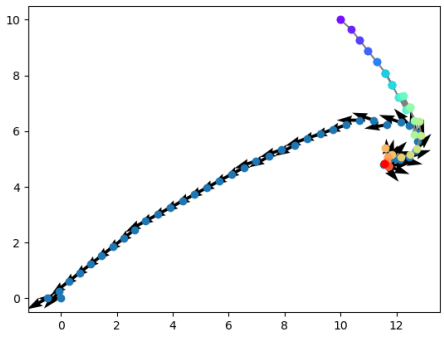
A first approach is to treat this as a **finite-horizon** problem, and hence minimize

$$\mathbb{E}\Big\{ g_N(\boldsymbol{d}_N) + \sum_{k=0}^{N-1} g_k(\boldsymbol{d}_k) \Big\} \tag{3.6}$$

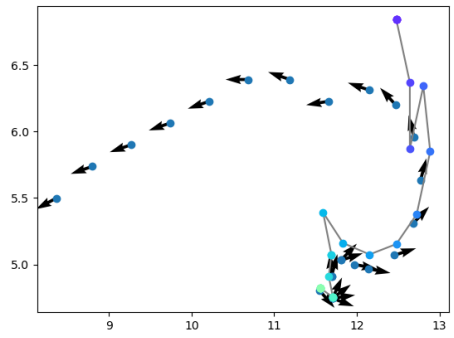
where $g_N(\boldsymbol{d}_N) = d_{N,x}^2 + d_{N,y}^2$.
From this point onward, we use the usual notation $\boldsymbol{x}_k = (x_k, y_k, \theta_k)$ for the system state, instead of $\boldsymbol{d}_k$. Whenever this could be confused with the vector $\boldsymbol{x}_k$ describing the vehicle position, we state the distinction explicitly. For the dynamic programming equation, we have $J_N(\boldsymbol{x}_N) = x_N^2 + y_N^2$ and

$$\begin{aligned}
J_N(\boldsymbol{x}_N) &= x_N^2 + y_N^2 \\
J_k(\boldsymbol{x}_k) &= \min_{\boldsymbol{u}_k} \quad \mathbb{E}_{w_k}\Big\{ g_k(\boldsymbol{x}_k) + J_{k+1}(f(\boldsymbol{x}_k, \boldsymbol{u}_k, w_k)) \Big\} \\
&= g_k(\boldsymbol{x}_k) + \min_{\boldsymbol{u}_k} \mathbb{E}_{w_k}\Big\{ J_{k+1}(f(\boldsymbol{x}_k, \boldsymbol{u}_k, w_k)) \Big\} \\
&= g_k(\boldsymbol{x}_k) + \min_{\boldsymbol{u}_k}\Big\{ \sum_{i=-1}^{1} p(w_k = i) \cdot J_{k+1}(f(\boldsymbol{x}_k, \boldsymbol{u}_k, w_k = i)) \Big\} \Rightarrow \\
J_k(\boldsymbol{x}_k) &= g_k(\boldsymbol{x}_k) + \frac{1}{3} \min_{\boldsymbol{u}_k}\Big\{ \sum_{i=-1}^{1} J_{k+1}(f(\boldsymbol{x}_k, \boldsymbol{u}_k, w_k = i)) \Big\}
\end{aligned} \tag{3.7}$$

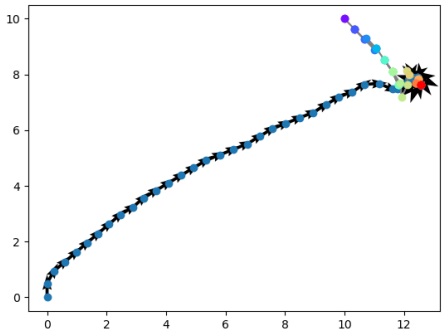
At each state $\boldsymbol{x}_k$, we therefore look N steps ahead and select a control $\boldsymbol{u}_k$. This procedure is repeated until the distance between the vehicle and the target falls below a threshold, $||(x_k - a_k, y_k - b_k)||_2 \leq threshold$. The following examples use different initial vehicle states $\boldsymbol{x}_0$ and target positions $\boldsymbol{r}_0$. Each case shows the entire trajectory until the algorithm terminates, together with the final 25 steps to make convergence clearer. In every case below, we consider $N = 2$ steps.



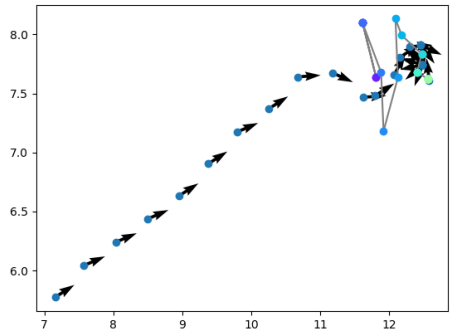
(a) $\boldsymbol{x}_0 = (0, 0, -\pi), \boldsymbol{r}_0 = (10, 10)$



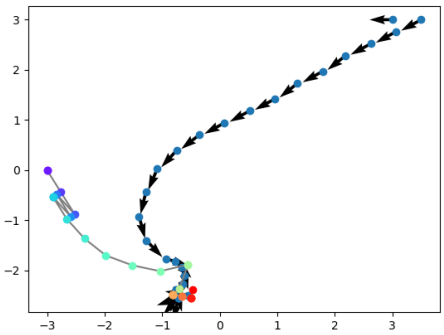
(b) Final 25 steps



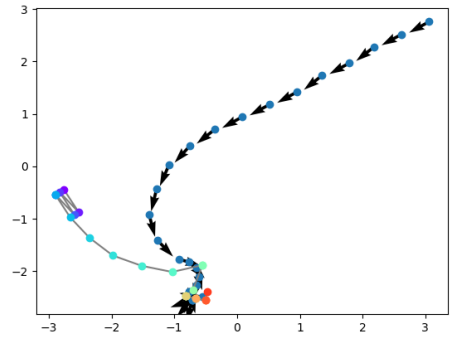
(c) $\boldsymbol{x}_0 = (0, 0, \frac{\pi}{2}), \boldsymbol{r}_0 = (10, 10)$



(d) Final 25 steps



(e) $\boldsymbol{x}_0 = (3, 3, -\pi), \boldsymbol{r}_0 = (-3, 0)$

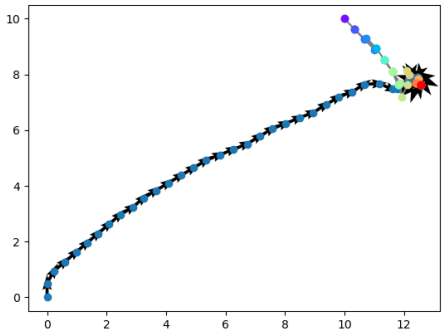


(f) Final 25 steps

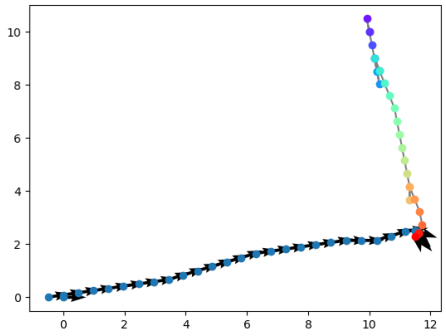
The algorithm does eventually converge to the target $\boldsymbol{r}$. In the simulation, the value of N appears to have little effect on the results. For example, the controls $\boldsymbol{u}_k$ are almost identical for $N = 2$ and $N = 4$. We next consider the case in which the three outcomes are not equally likely. For example, suppose that

$$P(w_k = i) = \begin{cases} 1/6 & i = -1 \\ 0 & i = 0 \\ 5/6 & i = 1 \end{cases}$$

To compare with the previous results, let $\boldsymbol{x}_0 = (0, 0, 0)$ and $\boldsymbol{r}_0 = (10, 10)$, as in panel (c) above, except that the vehicle previously started with orientation $\pi/2$. Below, we compare the behavior of the two systems when w_k takes values in $\{-1, 0, 1\}$ with probabilities $\{1/3, 1/3, 1/3\}$ and $\{1/6, 0, 5/6\}$, respectively.



(g) $\boldsymbol{p} = (1/3, 1/3, 1/3)$



(h) $\boldsymbol{p} = (1/6, 0, 5/6)$

The figures show that, even with $N = 2$, the controller can anticipate that the system will move downward in the future. It therefore predicts that moving toward the future target position is preferable to moving toward its current position.

3.2 Certainty equivalence

Although the preceding algorithm performs relatively well, evaluating the expectation in equation (3.7) has high computational complexity. At each step k, we must compute $J_{k+1}(f(\boldsymbol{x}_k, \boldsymbol{u}_k, w_k = i))$ for every $i \in \{-1, 0, 1\}$. As we move farther backward in time, increasing l, the complexity therefore grows exponentially, eventually making the problem infeasible. For this reason, we replace w_k by its mean, $\bar{w}_k = \mathbb{E}[w_k]$.

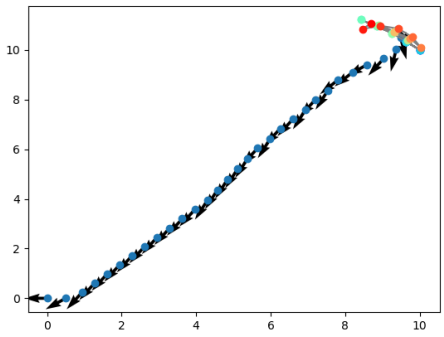
$$\bar{w}_k = \sum_{i=-1}^{1} i \cdot p(w_k = i) = 0$$

The error dynamics in equation (3.3) and the dynamic programming algorithm in equation (3.7) then become

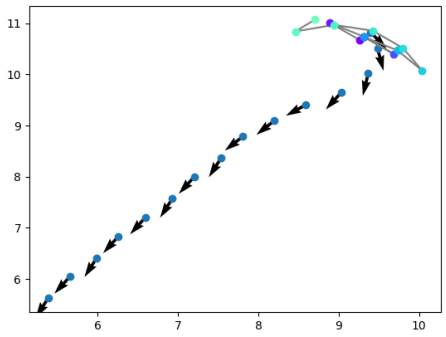
$$\boldsymbol{x}_{k+1} = f(\boldsymbol{x}_k, \boldsymbol{u}_k, w_k = 0) \Rightarrow \quad \boldsymbol{x}_{k+1} = \boldsymbol{x}_k + I_{3\times 3} \cdot \boldsymbol{y}_k$$

$$J_k(\boldsymbol{x}_k) = g_k(\boldsymbol{x}_k) + \min_{\boldsymbol{u}_k}\Big\{J_{k+1}\Big(f(\boldsymbol{x}_k, \boldsymbol{u}_k, w_k = 0)\Big)\Big\}$$

With this method, the computational complexity is N^3 times lower than when using equation (3.7), where N is the lookahead and 3 is the number of possible values of w_k. Only the counterpart of the first example, panel (a), is shown below, since the remaining cases are similar.



(i) $\boldsymbol{x}_0 = (0, 0, -\pi), \boldsymbol{r}_0 = (10, 10)$



(j) Final 25 steps

The results are very good in this case because w_k distributes its probability over very few values (only

three), and therefore has very small variance. Conversely, we expect the algorithm to perform worse as the variance increases.

4 Modeling a more challenging problem

The state vector is

$$\boldsymbol{x} = \begin{pmatrix} x \\ y \\ \dot{x} \\ \dot{y} \\ \theta \\ \dot{\theta} \end{pmatrix}$$

At each instant, the vehicle speed is $V = \sqrt{\dot{x}^2 + \dot{y}^2}$. The system input is $\boldsymbol{u} = \begin{pmatrix} u_1 & u_2 \end{pmatrix}^T$, whose components correspond to the throttle and steering inputs at each instant. A schematic of the vehicle model is shown below. The forward force applied to the vehicle is $F_u = F_{max} \cdot u_1$, while the turning

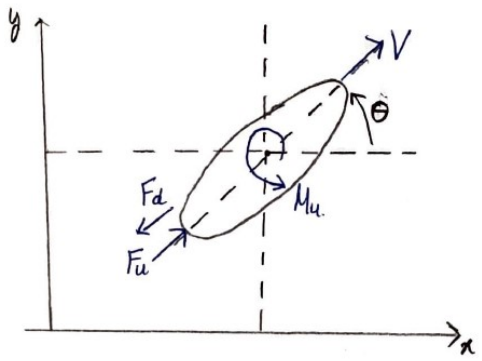


Figure 5: Vehicle motion model

moment is $M_u = M_{max} \cdot u_2$, with $u_i \in [-1, 1]$, $i = 1, 2$. To express the system dynamics in the form $\dot{\boldsymbol{x}} = f(\boldsymbol{x}(t), \boldsymbol{u}(t))$, we use Newton's equations of motion. Specifically,

$$\begin{aligned} F = ma \iff \begin{pmatrix} F_x \\ F_y \end{pmatrix} = m \begin{pmatrix} a_x \\ a_y \end{pmatrix} \implies \\ \begin{pmatrix} a_x \\ a_y \end{pmatrix} = \frac{1}{m} \begin{pmatrix} F_u \cos(\theta) - F_d \cos(\theta) \\ F_u \sin(\theta) - F_d \sin(\theta) \end{pmatrix} \implies \\ \begin{pmatrix} \ddot{x} \\ \ddot{y} \end{pmatrix} = \frac{1}{m} \begin{pmatrix} F_u \cos(\theta) - F_d \cos(\theta) \\ F_u \sin(\theta) - F_d \sin(\theta) \end{pmatrix} \end{aligned}$$

Similarly, for rotational motion,

$$M = I_b \ddot{\theta} \implies \ddot{\theta} = \frac{1}{I_b} \cdot M_u \tag{4.1}$$

where M is the total moment and I_b is the moment of inertia. There is also a force opposing the motion, $F_d = C_T V^2$, where C_T is a constant.
Using these expressions, we can write the differential equation governing the system:

$$\begin{pmatrix}\dot{x}\\ \dot{y}\\ \ddot{x}\\ \ddot{y}\\ \dot{\theta}\\ \ddot{\theta}\end{pmatrix} = \begin{pmatrix}\dot{x}_1\\ \dot{x}_2\\ \dot{x}_3\\ \dot{x}_4\\ \dot{x}_5\\ \dot{x}_6\end{pmatrix} = \begin{pmatrix} x_3 \\ x_4 \\ \frac{1}{m}\left[F_u\cos(x_5) - F_d\cos(x_5)\right] \\ \frac{1}{m}\left[F_u\sin(x_5) - F_d\sin(x_5)\right] \\ x_6 \\ \frac{1}{I_b}(M_u - M_d)\end{pmatrix} \qquad \begin{aligned} V &= \sqrt{x_3^2 + x_4^2} \\ F_d &= C_T V^2 \\ F_u &= F_{max} u_1 \\ M_u &= M_{max} u_2 \\ M_d &= C_r(\dot{\theta})^2 \end{aligned} \tag{4.2}$$

We have thus obtained the desired form $\dot{\boldsymbol{x}} = f(\boldsymbol{x}, \boldsymbol{u})$. Discretizing this equation gives

$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \Delta t \cdot f(\boldsymbol{x}_k, \boldsymbol{u}_k) \tag{4.3}$$

The state vector has six dimensions, making the methods used so far impractical. We therefore choose an actor–critic method to approximate the solution to this problem.
Specifically, this method is a variant of policy iteration in which, instead of computing the cost J_μ under policy μ using

$$J_\mu(\boldsymbol{x}) = g(\boldsymbol{x}, \mu(\boldsymbol{x})) + aJ_\mu\left(f(\boldsymbol{x}, \mu(\boldsymbol{x}))\right) \qquad \forall \boldsymbol{x} \tag{4.4}$$

we obtain an estimate $\tilde{J}_\mu$ of the cost of the current policy. This change is necessary because, previously (for example, in value iteration), the state space was discretized, allowing the cost of a policy to be computed analytically by solving a system of S equations, where S is the number of discrete states. Adapting the ideas of policy iteration, we modify the general method as follows:

1. Start with a random policy μ^0.

2. Compute the cost of the current policy without using equation (4.4). Instead, generate random samples $\boldsymbol{x}^s, s = 1, 2, \ldots, q$, and compute the cost at each state $\boldsymbol{x}^s$ under the current policy μ^k by taking L steps:

$$J^s_{\mu^k}(\boldsymbol{x}^s) = \sum_{l=0}^{L-1} a^l g(\boldsymbol{x}^s_l, \mu^k(\boldsymbol{x}^s_l)) \qquad \boldsymbol{x}^s_{l+1} = f(\boldsymbol{x}^s_l, \mu^k(\boldsymbol{x}^s_l)) \tag{4.5}$$

   This produces a dataset of pairs $(\boldsymbol{x}^s, J^s_{\mu^k})$, which can be used to train an algorithm to predict the cost $\tilde{J}_{\mu^k}(\boldsymbol{x})$ of the current policy at each point $\boldsymbol{x}$ in the state space. Choosing a learnable parametric form $\tilde{J}_{\mu^k}(\boldsymbol{x}, \boldsymbol{r}) = \phi_1^T(\boldsymbol{x}) \cdot \boldsymbol{r}_1$, we have

$$\hat{\boldsymbol{r}}_1 = \arg\min_{\boldsymbol{r}} \frac{1}{q} \sum_{s=1}^{q} \left(J^s_{\mu^k}(\boldsymbol{x}^s) - \phi_1^T(\boldsymbol{x}^s) \cdot \boldsymbol{r}_1\right)^2 \tag{4.6}$$

   where $\phi_1(\boldsymbol{x})$ is a transformation of the input. This model is linear in its parameters, so it can be fitted very quickly at each iteration using a least-squares method.

3. Having found $\tilde{J}_{\mu^k}$, improve the policy by finding a new policy μ^{k+1}. Since the state space is again very large, obtain samples $u^s, s = 1, 2, \ldots, q$ and approximate the new policy with $\tilde{\mu}^{k+1}(\boldsymbol{x}, \boldsymbol{r}) = \phi_2^T(\boldsymbol{x}) \cdot \boldsymbol{r}_2$, as before. Specifically, generate samples using

$$\boldsymbol{u}^s \in \arg\min_{\boldsymbol{u}} \left\{g(\boldsymbol{x}^s, \boldsymbol{u}) + a\tilde{J}_{\mu^k}(f(\boldsymbol{x}^s, \boldsymbol{u}))\right\} \tag{4.7}$$

   where $\tilde{J}_{\mu^k}$ is the function found previously. Then compute the parameter $\boldsymbol{r}$ of the new policy from

$$\hat{\boldsymbol{r}}_2 = \arg\min_{\boldsymbol{r}} \frac{1}{q} \sum_{s=1}^{q} \left|\left|\boldsymbol{u}^s - \phi_2^T(\boldsymbol{x}^s) \cdot \boldsymbol{r}_2\right|\right|^2 \tag{4.8}$$

Return to step 2 and repeat the procedure using the new policy $\tilde{\mu}^{k+1}$, the new cost $\tilde{J}_{\mu^{k+1}}$, and so on.

4.1 Simulation

As in the first part, we consider a target state $\boldsymbol{x}_{ref} = (x_{ref} = 10, y_{ref} = 10, \theta_{ref} = 0)$, ignoring the linear and angular velocities. Again, as in part 1, we assume that

$$g(\boldsymbol{x}_k) = (\boldsymbol{x}_k - \boldsymbol{x}_{ref})^T W_s (\boldsymbol{x}_k - \boldsymbol{x}_{ref})$$

where $\boldsymbol{x}_k$ includes only the x, y positions and the angle θ for the purpose of evaluating the cost. To generate samples as required in steps 2 and 3 of the method above, we sample each of the six variables independently and uniformly over intervals related to the fixed parameters characterizing the system in equation (4.2).
For each policy μ^k, we generate $q = 8000$ trajectories of length $L = 100$, then fit the corresponding function $\tilde{J}_{\mu^k}(\boldsymbol{x}, \boldsymbol{r}) = \phi_1^T(\boldsymbol{x}) \cdot \hat{\boldsymbol{r}}_1$.
We then proceed to step 3, generating $q = 4000$ control samples $\boldsymbol{u}^s$ and finding the new policy $\mu^{k+1}(\boldsymbol{x}) = \phi_2^T(\boldsymbol{x}) \cdot \boldsymbol{r}_2$.
Although the problem is not an infinite-horizon problem, since a specified terminal state can be reached because the system is controllable, we again treat it as an infinite-horizon problem, hence the constant a, because numerical instabilities may arise, especially for long trajectories.

Choice of basis functions $\phi_i(\boldsymbol{x})$ The simulation showed that the choice of bases $\phi_i(\boldsymbol{x}), i = 1, 2$, is particularly important for accurately approximating both the cost J_{μ^k} of each policy and the policy μ^k itself. A simple initial choice is a polynomial basis:

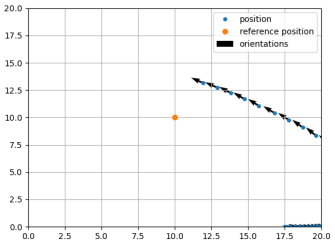
$$\phi_1(\boldsymbol{x}) = \phi_2(\boldsymbol{x}) = \begin{pmatrix} 1 & x_1 & x_2 & \dots & x_6 & x_1^2 & x_2^2 & \dots & x_6^2 \end{pmatrix}^T$$

With the features $\phi(\boldsymbol{x})$ available, we use the preceding actor–critic equations in the hope of improving the system behavior. However, approximating both μ^k and J_{μ^k} with functions that are linear in their parameters does not appear to work.
To confirm this finding, I selected another transformation of the data that appears more suitable for the problem. Specifically,

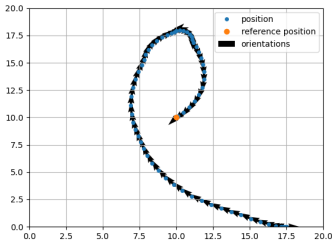
$$\phi(\boldsymbol{x}) = \begin{pmatrix} d & V & \cos\theta & \sin\theta & \dot{\theta} \end{pmatrix}^T \qquad d = \sqrt{(x - x_{ref})^2 + (y - y_{ref})^2} \qquad V = \sqrt{\dot{x}_1^2 + \dot{x}_2^2}$$

This transformation appears more relevant to our problem, since it directly encodes the distance from the target, the vehicle speed, and so on. Nevertheless, this transformation did not work either.
For these reasons, we use two separate neural networks to approximate the policy μ^k and the cost J_{μ^k}. The following page presents the results of this algorithm, showing the gradual improvement in our results.

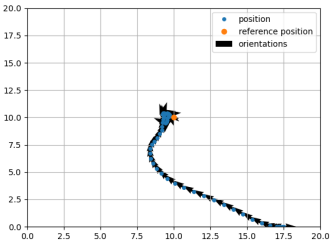
$$\mu^k(\boldsymbol{x}, \boldsymbol{r}) = \phi^T(\boldsymbol{x}) \cdot \boldsymbol{r} \tag{4.9}$$



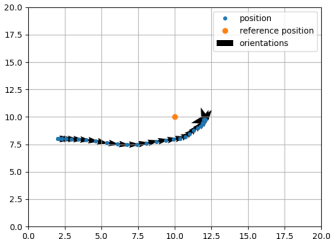
(a) 1st iteration



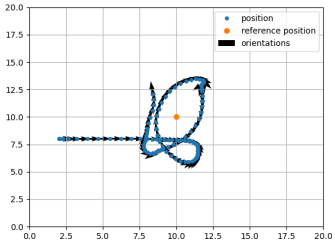
(b) 5th iteration



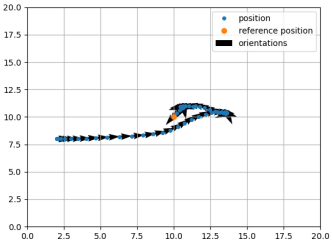
(c) 40th iteration



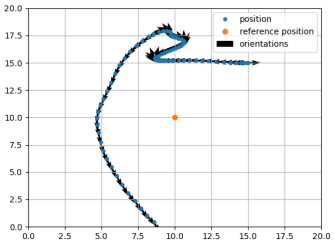
(d) 1st iteration



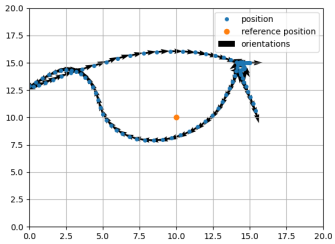
(e) 5th iteration



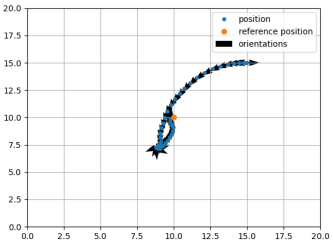
(f) 40th iteration



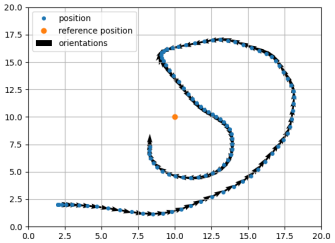
(g) 1st iteration



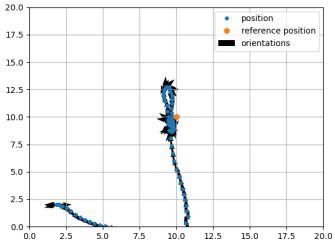
(h) 5th iteration



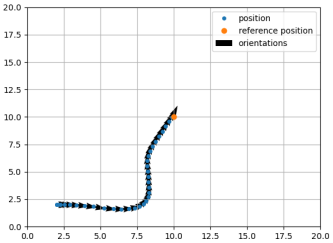
(i) 40th iteration



(j) 1st iteration



(k) 5th iteration



(l) 40th iteration

For the two networks to represent the policy and the cost satisfactorily, we ultimately had to use relatively large MLP architectures. The results above are not optimal, but algorithms of this kind, which use Monte Carlo methods to sample the state space, sometimes require very many iterations to converge.

References

[1] Dimitri P. Bertsekas, Reinforcement Learning and Optimal Control

[2] Dimitri P. Bertsekas, Dynamic Programming and Optimal Control volume 1

[3] Dimitri P. Bertsekas, Dynamic Programming and Optimal Control volume 2